

Customization of the CVA6 Core

Hanan Quispe

Institut Polytechnique de Paris
hanan.quispecondori@ip-paris.fr

Nicolas Derumigny

Inria
nicolas.derumigny@inria.fr

Abstract

This study investigates the performance and resource trade-offs of hardware sharing in the CVA6 RISC-V core, specifically targeting the Floating Point Unit (FPU). The research followed a three stage methodology: an initial microarchitectural design space exploration on a single core configuration to establish a baseline using UnixBench and NAS Parallel Benchmarks, the design of a custom arbitrator for shared FPU access, and a final evaluation of system performance. Critically, the hardware impact is assessed through resource utilization measuring FF, LUT, BRAM, and DSP. This approach quantifies the logic to performance trade-off, providing actionable insights for ASIC-targeted research within the OpenHW and PULP ecosystems.

1. Introduction

The optimization of performance, power, and area (PPA) through the sharing of underutilized execution units is a foundational strategy in multicore architecture. (Rodrigues et al.) establish a critical baseline for this approach by examining the sharing of floating-point (FP) datapaths between pairs of cores, an architecture similar to AMD's Bulldozer. Their findings highlight a distinct trade-off: while sharing can yield significant power savings, it may result in performance losses of up to 28%, though these losses can be halved by maintaining individual reservation stations while sharing only the execution units. This focus on efficiency is echoed in the RISC-V domain by (Islam and Kise), who demonstrate that sharing functional units in a five-stage scalar pipeline can reduce FPGA resource utilization—specifically saving up to 23.5% in LUT usage and 75% in DSP usage—with average performance drops ranging from 2.9% to 22.3%. Furthering the case for low-power applications, the Florian project (Park et al. a) and subsequent studies on IoT endnodes (Park et al. b) argue that dedicated FPUs are often "overkill" for

lightweight cores, proposing an external shared FPU that can achieve energy savings of nearly 80% in quad-core configurations. Complementing these architectural shifts, (Kumar et al.) explore switchable FPUs compliant with IEEE 754-2008, emphasizing that dynamic switching between integer and floating-point operations can significantly enhance computational efficiency and reduce execution latency.

However, scaling these resource-sharing techniques to an application-class core like the CVA6 (Sá et al.) introduces a level of complexity not addressed in simpler designs. The CVA6 is a significantly more complex core than what previous shared-FPU studies have utilized. It features a 6-stage, single-issue, out-of-order execution pipeline with an integrated FPU. Sharing its FPU between two cores raises sophisticated architectural questions, particularly regarding how to handle the scoreboard interaction, writeback arbitration, and pipeline stalls that do not arise with simpler in-order cores. Because CVA6 serves as the reference core for the OpenHW Group and is widely utilized in European research SoCs like the PULP ecosystem, this research is of high practical relevance.

2. Microarchitectural Design Space Exploration

Microarchitectural design space exploration necessitated a evaluation of both performance and resource utilization. To assess performance, we utilized standard workloads, specifically UnixBench and the NAS Parallel Benchmarks (NPB). Resource utilization was quantified using Vivado's post-implementation utilization reports.

2.1 Experimental Setup

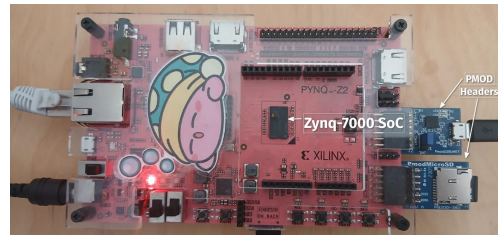


Figure 1. CVA6 Single Core deployment on a PYNQ-Z2 FPGA with PMOD Modules

Experiments were conducted on a PYNQ-Z2 FPGA, supplemented by two PMOD modules providing a microSD interface and a UART port for serial communication.

In addition to the setup described in figure 1 for automation purposes an additional Ethernet connection was used, resulting in the final experimental setup.

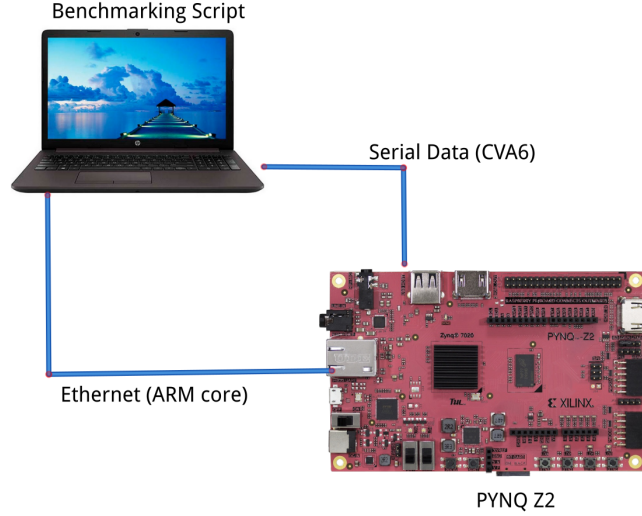


Figure 2. Experimental Setup

2.2 Resource Utilization Measurement

The first part of this project consisted in evaluating the resource usage due to different parameters of the CVA6 architecture. For evaluation purposes a baseline configuration is given with the following parameters.

Table 1. Baseline Bitstream Configuration

Parameter	Size
Data Cache	4096[bytes]
Data Cache Associativity	4
Data Cache Line Width	128
Instruction Cache	4096[bytes]
Instruction Cache Associativity	4
Instruction Cache Line Width	128
BTB Entries	16
BHT Entries	16

The CVA6 architecture was then implemented in a Pynq-Z2 FPGA. The resources of the FPGA can be mainly separated into 4 groups which are:

- Flip Flops (**FF**)
- Look up Tables (**LUT**)
- Digital Signal Processor (**DSP**)
- Block RAM (**BRAM**)

To start let's see the resource usage of the baseline bitstream.

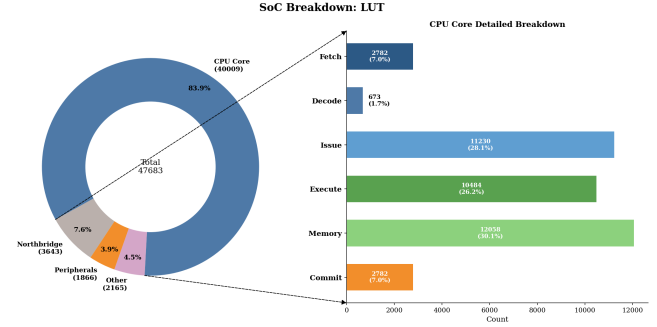


Figure 3. Baseline LUT Usage

Figure 3 illustrates that the baseline bitstream nearly saturates the Pynq-Z2's logic resources, with the CPU Core (83.9%) driven primarily by the heavy combinatorial requirements of the Memory (30.1%), Issue (28.1%), and Execute (26.2%) stages. Even with a lean 4KB cache, the 4-way associativity and 128-bit lines necessitate complex tag matching logic in the Memory stage, while the dual issue retirement width forces the scoreboard in the Issue stage to manage simultaneous instruction hazards, significantly increasing the LUT count. While the area efficient 16-entry branch predictor keeps the Fetch stage (7.0%) minimal, the overall total of 47,683 LUTs confirms that this CVA6 configuration is pushing the absolute limits of the Zynq-7020 hardware, leaving almost no room for additional logic without scaling back core features.

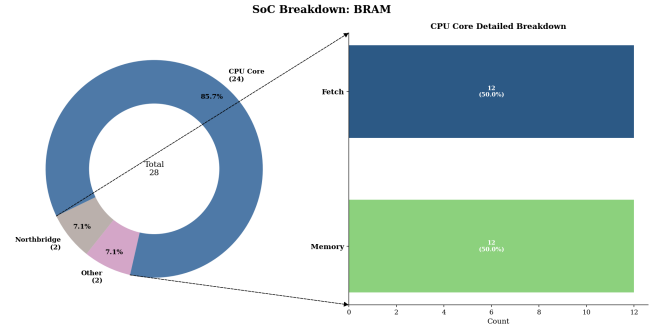


Figure 4. Baseline BRAM Usage

Figure 4 shows a total of 28 blocks, with the CPU Core claiming 85.7% (24 blocks) and the remaining 14.3% split equally between the Northbridge and "Other" logic. Within the core, resources are distributed in a perfectly symmetrical 50/50 split between the Fetch (12 blocks) and Memory (12 blocks) stages, while the Decode, Issue, Execute, and Commit stages utilize zero BRAM. This distribution reflects your 4KB cache configurations; the wide 128-bit lines and 4-way associativity require these blocks for both data and tag storage in the Instruction and Data caches. Notably, your 16-entry branch predictor is small enough to be synthesized into

LUTs rather than BRAM, contributing to a very low overall memory footprint that leaves 80% of the Zynq-7020's BRAM available for future expansion.

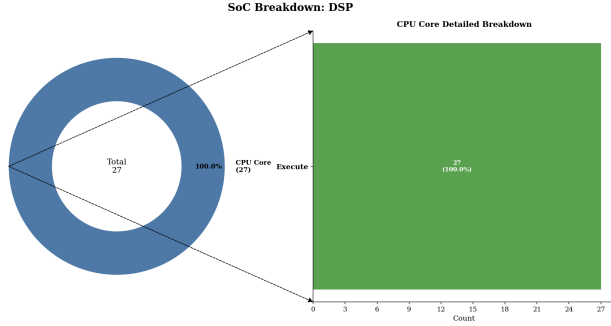


Figure 5. Baseline DSP Usage

Figure 5 shows that 100.0% of the 27 utilized DSP slices are concentrated within the CPU Core, specifically dedicated to the Execute (100.0%) stage. This absolute concentration is expected, as DSP slices are the specialized hardware primitives used to implement the high speed multipliers, dividers, and the Floating Point Unit (FPU) within the CVA6 pipeline.

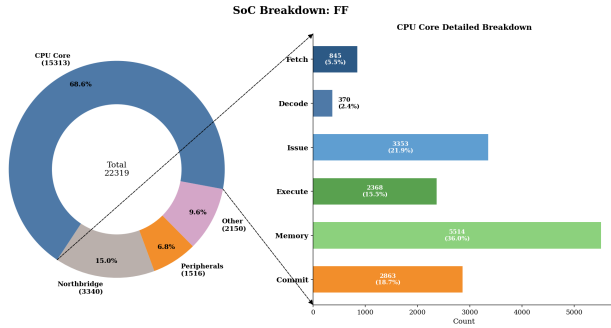


Figure 6. Baseline FF Usage

Figure 6 reveals a total of 22,319 Flip-Flops, where the CPU Core accounts for 68.6% (15,313 FF) of the state-holding elements, a lower share than its LUT dominance which indicates that components like the Northbridge (15.0%) require significant buffering for the AXI interconnect. Inside the core, the Memory stage (36.0%) is the largest consumer, housing the wide 128-bit registers needed to pipeline data from the caches and MMU. The Issue (21.9%) and Commit (18.7%) stages also show high utilization, representing the physical storage for the scoreboard's dependency tracking and the Reorder Buffer (ROB) required to maintain state for dual-issue retirement. This distribution highlights that while the logic is concentrated in execution, the "memory" of the processor—its pipeline registers and buffers—is distributed across the stages responsible for managing data flow and instruction order.

For visualization purposes the following fields have been included in the *other* category:

- **SPI Controller:** The interface used for the SD Card or Flash memory.
- **UART:** The serial interface used for the console/terminal.
- **CLINT (Core Local Interruptor):** The timer module responsible for generating software and timer interrupts.
- **BootROM:** The small Read-Only Memory containing the very first instructions the CPU executes upon reset.
- **Memory Interconnect:** The AXI adapters and shims that handle width or clock conversion between the CPU and the main memory bus.
- **Glue Logic:** The calculated remainder (SoC Total Everything Else). This represents the top-level wires, clock buffers, and reset synchronizers that connect all these modules together.

Also the calculation of the 6 stages involved the following modules from the utilization reports obtained from Vivado.

Table 2. Utilization Report Modules

Stage	Modules
Fetch	i_frontend + i_cva6_icache
Decode	id_stage_i
Issue	issue_stage_i
Execute	ex_stage_i
Memory	lsu_i + i_wt_dcache
Commit	i_scoreboard

2.3 Performance measurement

Performance measurement is based on the work by (Domingos et al.) as their work enabled hardware performance counter usage. For our setup the following counters are available:

- Number of Instructions
- Number of cycles

Those two counters enabled us to use the instruction per cycle (*IPC*) metric.

$$IPC = \frac{\text{Number of Instructions}}{\text{Number of cycles}} \quad (1)$$

Additionally Unixbench and NBP benchmark scores are reported too.

- **NPB: Mop/s (Millions of Operations per Second)**
- **Unixbench: Loops per second (lps), KB/sec, and operations per second**

Moreover, each benchmark was executed 5 times to ensure statistical significance in the measurements.

2.4 Parameter Sweeps

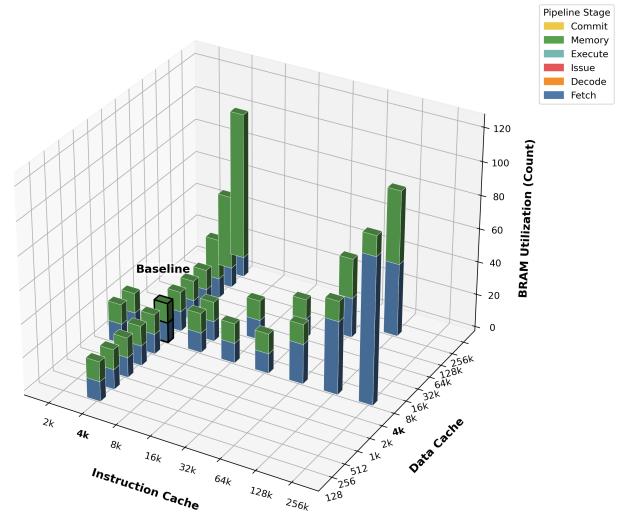
The following baseline parameters were established to evaluate the impact of microarchitectural configurations on performance and resource utilization.

Parameter	Size	CVA6 Stage
Data Cache	4096[<i>bytes</i>]	Memory
Data Cache Associativity	4	Memory
Data Cache Line Width	128	Memory
Instruction Cache	4096[<i>bytes</i>]	Fetch
Instruction Cache Associativity	4	Fetch
Instruction Cache Line Width	128	Fetch
Branch Target Buffer (BTB) Entries	16	Fetch
Branch History Table (BHT) Entries	16	Fetch
Retirement Width of the CPU	2	Commit

Sweep	Range	CVA6 Stage
Data Cache	$128 \rightarrow 256k[bytes]$	Memory
Data / Instruction	$1 \rightarrow 16$	Memory
Cache Associativity		Fetch
Instruction Cache	$2k \rightarrow 256k[bytes]$	Fetch
BTB Entries	$8 \rightarrow 32k$	Fetch
BHT Entries	$16 \rightarrow 16k$	Fetch

[illegible]

suggest that doubling or tripling your cache size would provide a massive ROI for system responsiveness without saturating the FPGA's remaining memory blocks.



2026/3/19

stacks grow proportionally as Instruction and Data cache capacities are increased. The plot highlights the "resource cliffs" inherent to the Zynq-7020; while the 4k/4k baseline is conservative, moving toward the 256k/256k corner of the graph would completely saturate the hardware's 140 available BRAM blocks. This visualization is critical for identifying which configurations allow for performance gains without preventing the inclusion of other logic-heavy features.

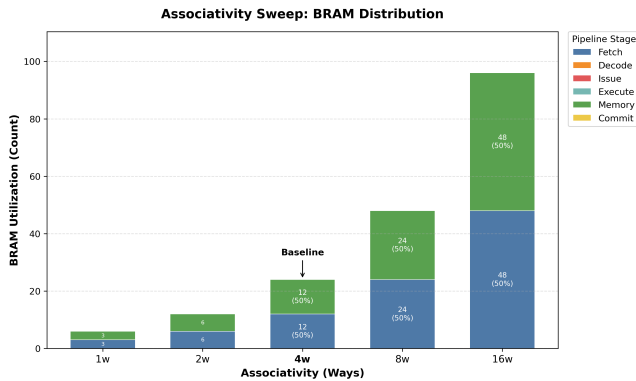


Figure 10. Associativity Sweep Utilization

The Associativity Sweep: BRAM Distribution plot reveals a strictly linear scaling between cache associativity and memory resources; doubling the "ways" from your 4-way baseline to 8 or 16 ways directly doubles the BRAM count to 48 and 96 blocks, respectively. The perfectly symmetrical distribution between the Fetch (blue) and Memory (green) stages indicates that increasing associativity impacts the Instruction and Data caches identically. While 16-way associativity provides better hit rates, it consumes nearly 70% of the Pynq-Z2's 140 BRAMs just for the caches, suggesting that the 4-way or 8-way configuration is likely the "sweet spot" for balancing performance against the remaining FPGA resource budget.

2.5 Contributions

- PR to the NPB repository
<https://github.com/benchmark-subsetting/NPB3.0-omp-C/pull/2>

- Automated tool for benchmarking architectural changes on the PYNQ Z2
- Baseline performance / utilization measurements for CVA6 multi-core

References

- R. Rodrigues, I. Koren, and S. Kundu, "Does the sharing of execution units improve performance/power of multicores?" vol. 14, no. 1, pp. 1–24. [Online]. Available: <https://dl.acm.org/doi/10.1145/2680543>
- M. A. Islam and K. Kise, "Efficient resource shared RISC-v multicore processor," in *2021 IEEE 14th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSoc)*, pp. 366–372. [Online]. Available: <https://ieeexplore.ieee.org/document/9691969/>
- J. Park, K. Han, E. Choi, S. Lee, J.-J. Lee, W. Lee, and M. Pedram, "Florian: Developing a low-power RISC-v multicore processor with a shared lightweight FPU," in *2023 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*, pp. 1–6. [Online]. Available: <https://ieeexplore.ieee.org/document/10244431/>
- J. Park, K. Han, E. Choi, J.-J. Lee, K. Lee, W. Lee, and M. Pedram, "Designing low-power RISC-v multicore processors with a shared lightweight floating point unit for IoT endnodes," vol. 71, no. 9, pp. 4106–4119. [Online]. Available: <https://ieeexplore.ieee.org/document/10602782/>
- V. Kumar, S. Goyal, A. Mishra, and K. B. J.K., "32-bit RISC-v processor with switchable floating point ALU," in *2025 IEEE 5th International Conference on VLSI Systems, Architecture, Technology and Applications (VLSI SATA)*, pp. 1–6. [Online]. Available: <https://ieeexplore.ieee.org/document/11070011/>
- B. Sá, L. Valente, J. Martins, D. Rossi, L. Benini, and S. Pinto, "CVA6 RISC-v virtualization: Architecture, microarchitecture, and design space exploration," vol. 31, no. 11, pp. 1713–1726. [Online]. Available: <https://ieeexplore.ieee.org/document/10233226/>
- J. M. Domingos, T. Rocha, N. Neves, N. Roma, P. Tomás, and L. Sousa, "Supporting RISC-v performance counters through linux performance analysis tools," in *2023 IEEE 34th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*, pp. 94–101, ISSN: 2160-052X. [Online]. Available: <https://ieeexplore.ieee.org/document/10265733/>

